

Here is a clean, structured, exam-oriented explanation of **Servlets: Dynamic Content Generation** — easy for revision and interviews.

Servlets: Dynamic Content Generation — Full Notes

1. What is a Servlet?

A **Servlet** is a Java program that runs on a web server and generates **dynamic web content** (like HTML, JSON, XML, text, etc.).

It processes client requests (usually from a browser) and sends customized responses.

Key Points

- Part of **Java EE / Jakarta EE**.
 - Runs inside a **Servlet Container** (Tomcat, Jetty, WebSphere, etc.).
 - Uses **HTTP** as the communication protocol.
 - Replaces older technologies like CGI (Common Gateway Interface).
-

2. Why Do We Need Servlets for Dynamic Content?

Static pages (HTML/CSS) are fixed — same content for everyone.

Dynamic content changes based on:

- User input (login result, form data)
- Database values (show products, orders)
- Session data (shopping cart)
- Business logic (calculations, processing)

Servlets generate content at runtime using request parameters, cookies, sessions, and database interaction.

3. How Servlets Generate Dynamic Content

3.1 Steps in Dynamic Content Generation

1. **Browser sends a request** (HTTP GET/POST)
2. **Servlet container forwards request** to a servlet

3. Servlet processes request

- read form parameters
- connect to DB
- apply business logic

4. Servlet generates output dynamically

5. Response sent back to browser

4. Servlet Lifecycle (Important for Interviews)

1. init()

- Called once when servlet loads
- Used for initialization (DB connections, reading config)

2. service()

- Called for each request
- Delegates to:
 - doGet()
 - doPost()
 - doPut()
 - delete()

3. destroy()

- Called once before servlet unloads
 - Release resources
-

5. Example: Simple Dynamic Content Using a Servlet

Dynamic Servlet Code

java

 Copy code

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class WelcomeServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        res.setContentType("text/html");
```

```
    PrintWriter out = res.getWriter();

    // dynamic data
    String name = req.getParameter("name");

    out.println("<h2>Welcome, " + name + "!</h2>");
    out.println("<p>Current Time: " + new java.util.Date() + "</p>");

    out.close();
}
}
```

This servlet dynamically:

- ✓ Reads user input (name)
- ✓ Shows current time
- ✓ Generates HTML dynamically

6. How Dynamic Content Is Delivered

Servlet uses:

- **PrintWriter** → for HTML text
- **OutputStream** → for binary data (PDF, image, file download)
- **JSON/XML** → for REST responses

Example: Dynamic JSON

java

 Copy code

```
res.setContentType("application/json");
out.println("{ \"status\": \"success\" }");
```

7. Techniques for Dynamic Content in Servlets

7.1 Request Parameters

Reading form inputs:

java

 Copy code

```
String username = req.getParameter("username");
```

7.2 Database Integration

Using JDBC inside servlets to fetch dynamic results.

7.3 Sessions

Personalized content for logged-in users:

java

Copy code

```
HttpSession session = req.getSession();
session.setAttribute("user", "Yuvraj");
```

7.4 Cookies

Customized experience:

java

Copy code

```
Cookie c = new Cookie("theme", "dark");
res.addCookie(c);
```

7.5 Redirects & Forwards

- `sendRedirect()` – client-side redirect
- `RequestDispatcher.forward()` – server-side forward

8. Servlet vs JSP for Dynamic Content

Feature	Servlet	JSP
Best use	Business logic	UI / View
Syntax	Pure Java	HTML + Java
Performance	Fast, compiled	Slight overhead
Maintenance	Harder for UI	Easier

Ideal Approach = Servlet (Controller) + JSP (View) → MVC Pattern

9. Advantages of Dynamic Content Using Servlets

- Faster than CGI (multi-threaded)
 - Platform independent (Java)
 - Secure (sessions, filters)
 - Can generate any type of content (not only HTML)
 - Scalable and robust
-

10. Interview Questions (Important)

1. How does a servlet generate dynamic content?
 2. Explain servlet lifecycle methods.
 3. What is the role of `service()`, `doGet()`, `doPost()` ?
 4. How is a servlet different from CGI?
 5. How do servlets handle sessions for dynamic content?
 6. Why do we use JSP with servlets?
-

If you want, I can also prepare:

- ✓ Short notes for quick revision
- ✓ Full handwritten-style notes
- ✓ Servlet interview Q/A set
- ✓ A mini project using servlets + JDBC

Just tell me!



Advantages of Servlets over CGI short notes

Here are **short, crisp, exam-ready notes** on **Advantages of Servlets over CGI**:

Advantages of Servlets over CGI (Short Notes)

1. Better Performance

- Servlets use a **multi-threaded model**, while CGI creates a **new process for every request**, which is slow.
2. **Low Memory & Resource Usage**
 - Threads in servlets are lightweight; CGI processes consume more memory.
 3. **Faster Request Handling**
 - Since servlets run inside the JVM and reuse the same instance, they respond much faster than CGI programs.
 4. **Platform Independent**
 - Servlets are written in Java → **portable** across all OS and servers running JVM.
 5. **Server-Side Caching**
 - Servlet container keeps the servlet **in memory**, enabling reuse and caching.
 6. **Robust & Secure**
 - Java provides **exception handling, garbage collection, security API**, making servlets more reliable than CGI scripts.
 7. **Easy to Maintain**
 - Servlets are modular, object-oriented, and easier to debug compared to CGI scripts (often written in C/Perl).
 8. **Better Scalability**
 - Multi-threading allows a servlet to handle **thousands of requests** efficiently, unlike CGI's one-process-per-request mode.
 9. **Rich API Support**
 - Built-in support for sessions, cookies, headers, database connections, etc., unlike CGI where you code everything manually.

If you want, I can also give:

- ✓ 2-mark version
- ✓ Long answer (5/10 marks)
- ✓ Servlet vs CGI comparison table



Servlet Life cycle
and ServletAPI&Deployment exam-oriented explanation
with questions

Below is a **clean, exam-oriented explanation** of **Servlet Life Cycle** and **Servlet API & Deployment**, along with **important exam questions**.

1. Servlet Life Cycle — Exam-Oriented Explanation

The **Servlet Life Cycle** defines how a servlet is created, used, and destroyed by the servlet container (Tomcat/Jetty).

A servlet goes through **3 main phases**:

(1) Loading & Instantiation

- Container loads the servlet class into memory.
 - Creates **one instance** of the servlet.
 - Happens only **once** when the servlet is first requested (or at server startup if load-on-startup is used).
-

(2) Initialization — `init()`

- Container calls `init(ServletConfig config)`.
- Executes only **one time** in the servlet's lifetime.
- Used for:
 - Initial setup
 - DB connections
 - Reading configuration from `web.xml`

Prototype:

java

 Copy code

```
public void init() throws ServletException
```

(3) Request Handling — `service()`

- Called for **every client request**.
- Container passes `HttpServletRequest` and `HttpServletResponse`.
- `service()` internally calls:

- `doGet()` → for GET
- `doPost()` → for POST
- `doPut()` , `doDelete()` etc.

Prototype:

java

 Copy code

```
public void service(HttpServletRequest req, HttpServletResponse res)
```

Important point:

Only **one servlet object**, but **multiple threads** are created for each request.

(4) Destruction — `destroy()`

- Called once when container unloads the servlet or server shuts down.
- Used to release resources:
 - Closing DB connections
 - Stopping background threads

Prototype:

java

 Copy code

```
public void destroy()
```

Full Servlet Life Cycle Diagram (Simple)

1. Load Class → Instantiate → `init()`
 2. `service()` (multiple times for requests)
 3. `destroy()`
-

2. Servlet API — Exam-Oriented Explanation

Servlet API is provided by **Jakarta EE** (previously Java EE).

It contains classes and interfaces needed to build servlets.

Important Packages

1. jakarta.servlet

Contains general servlet interfaces:

- Servlet
- ServletRequest
- ServletResponse
- ServletConfig
- ServletContext
- RequestDispatcher
- GenericServlet

2. jakarta.servlet.http

Used for HTTP-based servlets:

- HttpServlet
- HttpServletRequest
- HttpServletResponse
- HttpSession
- Cookie

Important Interfaces

(1) Servlet

Base interface for all servlets. Defines:

- `init()`
- `service()`
- `destroy()`
- `getServletInfo()`
- `getServletConfig()`

(2) ServletRequest

Used to read:

- request parameters
- headers
- attributes
- body data

Example:

java

 Copy code

```
String name = req.getParameter("name");
```

(3) ServletResponse

Used to send output to the client:

- Setting content type
- Getting output stream / writer

Example:

java

 Copy code

```
res.setContentType("text/html");  
PrintWriter out = res.getWriter();
```

Important Classes

1. GenericServlet

- Protocol-independent servlet class.
- Implements Servlet interface.
- Used for **non-HTTP** servlets.

2. HttpServlet

- Most commonly used.
- Provides:
 - doGet()
 - doPost()
 - doPut()
 - delete()

3. Servlet Deployment — Exam-Oriented Explanation

Servlets must be **deployed** in a servlet container (Tomcat).

Two Ways to Deploy a Servlet

(1) Deployment using web.xml (traditional)

Location:

WEB-INF/web.xml

Example:

xml

 Copy code

```
<servlet>
  <servlet-name>hello</servlet-name>
  <servlet-class>com.demo.HelloServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>hello</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

(2) Deployment using Annotations (modern)

Inside servlet class:

java

 Copy code

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet { }
```

Advantages:

- No need for web.xml
- Cleaner and faster development

4. Exam-Oriented Questions (Most Asked)

Short Answer / 2 Marks

1. What is servlet life cycle?
2. Write the purpose of `init()` and `destroy()` methods.
3. What is the difference between `GenericServlet` and `HttpServlet`?

4. What is the use of `ServletConfig` ?
 5. What is deployment descriptor?
-

Long Answers / 5–10 Marks

1. Explain the servlet life cycle with a neat diagram.
 2. Describe the Servlet API and its major interfaces.
 3. Explain `HttpServletRequest` and `HttpServletResponse`.
 4. Explain servlet deployment using `web.xml` and annotations.
 5. What are the responsibilities of a servlet container?
-

Viva / Interview Questions

1. Why is `service()` called multiple times but `init()` only once?
 2. What is the difference between `doGet()` and `doPost()` ?
 3. How many instances of a servlet are created?
 4. Why are servlets multithreaded?
 5. What is the difference between `ServletContext` and `ServletConfig`?
-

If you want, I can also prepare:

- ✓ Handwritten-style notes
- ✓ 5-mark answer format
- ✓ Servlet API summary table
- ✓ Diagram-based notes



Servlet API & Deployment ,exam-oriented explanation
with important exam questions.

Here is a **clear, concise, exam-oriented** explanation of **Servlet API & Deployment**, along
with the **most important exam questions**.

✓ Servlet API — Exam-Oriented Explanation

The **Servlet API** is a set of classes and interfaces provided by **Jakarta EE** (formerly Java EE) used to create, configure, and manage servlets.

It mainly contains two important packages:

1 jakarta.servlet Package

It contains general interfaces and classes for ALL servlets (protocol-independent).

Important Interfaces

✓ Servlet

- Base interface for all servlets.
- Contains lifecycle methods: `init()`, `service()`, `destroy()`.

✓ ServletRequest

Used to **read client request data**:

- Form data (parameters)
- Header information
- Client IP
- Attributes

java

Copy code

```
String user = req.getParameter("username");
```

✓ ServletResponse

Used to **send output** to the client:

- Set content type
- Get output stream / writer

java

Copy code

```
res.setContentType("text/html");  
PrintWriter out = res.getWriter();
```

✓ ServletConfig

- Provides **initialization parameters** for a servlet.

- Defined in `web.xml` .

✓ ServletContext

- Shared for the whole application.
 - Used for global parameters, logging, and accessing resources.
-

2 jakarta.servlet.http Package

Contains HTTP-specific classes.

Important Classes

✓ HttpServlet

Most commonly used class.

Provides methods for each HTTP request:

- `doGet()`
- `doPost()`
- `doPut()`
- `doDelete()`

✓ HttpServletRequest

Used to read:

- Query parameters
- Form data
- Cookies
- Headers

✓ HttpServletResponse

Used to send:

- HTML
- JSON
- Redirect
- Cookies

✓ HttpSession

Used for storing user session data (login, cart, etc.).

✓ Cookie

Represents a browser cookie.

3 Important Supporting Interfaces

✔ RequestDispatcher

Used to:

- Forward a request
- Include another resource

java

```
RequestDispatcher rd = req.getRequestDispatcher("home.jsp");
rd.forward(req, res);
```

Copy code

Summary Table (Useful for Exams)

Component	Purpose
Servlet	Defines servlet lifecycle
ServletRequest	Read client data
ServletResponse	Send output
ServletConfig	Per-servlet config
ServletContext	App-level info
HttpServlet	HTTP-based servlet
HttpServletRequest	HTTP request info
HttpServletResponse	HTTP response handling
HttpSession	Manage user sessions
Cookie	Client-side state

✓ Servlet Deployment — Exam-Oriented Explanation

Deployment means **registering a servlet** so the container knows:

- The servlet class
- The URL pattern

Two deployment methods:

1 Deployment Using `web.xml` (Deployment Descriptor)

Located in:

`WEB-INF/web.xml`

Servlet registration format:

xml

 Copy code

```
<servlet>
  <servlet-name>Hello</servlet-name>
  <servlet-class>com.demo.HelloServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>Hello</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

Advantages

- Centralized configuration
- Supported by all servlet containers

2 Deployment Using Annotations (Modern Method)

No need to edit `web.xml`.

Use `@WebServlet` above the class.

java

 Copy code

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet { }
```


Advantages

- Less code
- Faster development
- Easier for beginners

Folder Structure of a Web Application

vbnet

 Copy code

```
MyApp/  
├─ index.html  
├─ WEB-INF/  
│   ├── web.xml  
│   └─ classes/  
│       └─ com/example/*.class  
└─ lib/*.jar
```

Responsibilities of the Servlet Container

(Frequently asked)

- Load and instantiate servlets
- Manage lifecycle
- Provide multithreading
- Provide request/response objects
- Handle security
- Manage sessions
- URL mapping

MOST IMPORTANT EXAM QUESTIONS

Short Answers (2 Marks)

1. What is Servlet API?
2. What is the use of ServletConfig?
3. Define ServletContext.
4. What is the purpose of RequestDispatcher?

5. What is a Deployment Descriptor?

Long Answers (5 Marks)

1. Explain Servlet API and its major interfaces.
 2. What are `HttpServletRequest` and `HttpServletResponse`?
 3. Explain servlet deployment using `web.xml` with example.
 4. Describe servlet mapping and URL pattern.
-

Long Answers (10 Marks)

1. Explain all components of the Servlet API with examples.
 2. Explain different methods of servlet deployment.
 3. Describe the architecture of a servlet-based web application.
-

Viva / Interview Questions

1. Difference between `ServletConfig` and `ServletContext`?
 2. Why do we use `@WebServlet` ?
 3. What is servlet mapping?
 4. What is the role of `HttpServlet` class?
 5. What is the use of `RequestDispatcher`?
-

If you want, I can also prepare:

- ✓ Handwritten-style notes
- ✓ 5-mark or 10-mark answer format
- ✓ Servlet API short notes (one page)



Servlet Annotations exam-oriented explanation with important exam questions.

Below is a **clear, exam-oriented explanation** of **Servlet Annotations**, followed by the **most important exam questions**. This is perfect for university theory exams, viva, and quick revision.

✓ Servlet Annotations — Exam-Oriented Explanation

Servlet annotations were introduced in **Servlet 3.0** to simplify servlet configuration. They allow developers to **configure servlets directly in Java code**, without editing the `web.xml` file.

The most commonly used annotations are:

1 @WebServlet

Used to declare a servlet and map it to a **URL pattern**.

Syntax

java

 Copy code

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet { }
```

Features

- Replaces traditional `<servlet>` and `<servlet-mapping>` in `web.xml`
- Supports multiple URL mappings
- Allows init parameters

Example with Init Params

java

 Copy code

```
@WebServlet(
    urlPatterns = {"/hello", "/greet"},
    initParams = {
        @WebInitParam(name="user", value="Admin")
    }
)
public class HelloServlet extends HttpServlet { }
```

2 @WebInitParam

Used to pass **initialization parameters** to a servlet.

Example

java

 Copy code

```
@WebInitParam(name="email", value="admin@example.com")
```

These can be accessed using:

java

 Copy code

```
String email = getServletConfig().getInitParameter("email");
```

3 @WebFilter

Used to define a **filter**, which intercepts requests and responses.

Example

java

 Copy code

```
@WebFilter("/secure/*")  
public class AuthFilter implements Filter { }
```

Purpose of Filters

- Authentication
- Logging
- Compression
- Request/response modification

4 @WebListener

Used to register **listeners** (objects that respond to events).

Types of events

- Application lifecycle

- Session creation/destruction
- Request lifecycle

Example

java

 Copy code

```
@WebListener  
public class AppListener implements ServletContextListener { }
```

5 @MultipartConfig

Used when uploading files using servlets.

Example

java

 Copy code

```
@MultipartConfig  
@WebServlet("/upload")  
public class UploadServlet extends HttpServlet { }
```

Features

- Controls max file size
- Enables multi-part/form-data processing

Why Servlet Annotations Are Useful? (Exam Points)

- Eliminates the need for `web.xml`
- Reduces configuration time
- More readable and maintainable
- Allows configuration close to Java code
- Makes deployment faster and more flexible

Example Program Using Annotation (Exam-Friendly)

java

 Copy code

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h2>Hello from Annotation-based Servlet!</h2>");
    }
}
```



Important Annotation Attributes (must write in exam)



Attributes of @WebServlet

- urlPatterns
- name
- initParams
- loadOnStartup



Attributes of @MultipartConfig

- maxFileSize
- maxRequestSize
- fileSizeThreshold



Most Important Exam Questions

2-Mark / Short Answer Questions

1. What is the purpose of @WebServlet annotation?
2. Define @WebFilter and its use.
3. What is the @MultipartConfig annotation?
4. Write any two advantages of servlet annotations.
5. What is the role of @WebListener ?

5-Mark Questions

1. Explain different types of servlet annotations with examples.
 2. Write a servlet program using annotations to display a simple message.
 3. What is the difference between annotation-based and XML-based servlet configuration?
 4. Explain the purpose and usage of `@WebServlet` and `@WebInitParam`.
-

10-Mark / Long Answer Questions

1. Describe in detail all servlet annotations with syntax and examples.
 2. Discuss annotation-based configuration in servlets. Compare it with `web.xml` deployment.
 3. Explain the working of filters and listeners using annotation-based configuration.
-

Viva / Oral Questions

1. Why were annotations introduced in servlets?
 2. Can we use both annotations and `web.xml` together?
 3. What is the use of the `loadOnStartup` attribute?
 4. How to configure multiple URL patterns in annotations?
-

If you want, I can also provide:

- ✓ One-page short notes
- ✓ Diagram-based notes
- ✓ Side-by-side comparison: XML vs Annotation configuration



The Servlet interface

The `HttpServlet`, `HttpServletRequest`, `HttpServletResponse` exam-oriented explanation with important exam questions.

Below is a **clean, exam-oriented** explanation of

The Servlet Interface, `HttpServlet`, `HttpServletRequest`, and `HttpServletResponse`,

along with **important exam questions** — perfect for theory exams and viva.

✓ 1. The Servlet Interface — Exam-Oriented Explanation

The **Servlet interface** is the **core** of the Servlet API.

Every servlet **must** implement this interface (directly or indirectly).

It defines **5 methods**, which form the **servlet life cycle**:

Methods of Servlet Interface

1 init(ServletConfig config)

- Called **once** when the servlet is first created.
 - Used for initialization (DB connections, reading init params).
-

2 service(ServletRequest req, ServletResponse res)

- Called **for every request**.
 - Contains the main request-processing logic.
-

3 destroy()

- Called **once** when servlet is removed or server shuts down.
 - Used to release resources.
-

4 getServletConfig()

- Returns the **ServletConfig** object.
-

5 getServletInfo()

- Returns descriptive information about the servlet (author, version).
-

Class Hierarchy (Important for exams)

kotlin

```
Servlet (interface)
  ↑
GenericServlet (abstract class)
  ↑
HttpServlet (abstract class)
```

✓ 2. HttpServlet — Exam-Oriented Explanation

`HttpServlet` is an **abstract class** used to create servlets that handle **HTTP requests**.

It extends `GenericServlet` and provides **`doXxx()` methods** for each HTTP operation.

Commonly Used Methods

✓ `doGet()`

Handles HTTP GET requests
(used for fetching/displaying data)

✓ `doPost()`

Handles HTTP POST requests
(used for form submission, sending data to server)

✓ `doPut()`, `doDelete()`

Used in RESTful APIs.

✓ `service()`

You normally do **not override** this.
It internally calls the appropriate `doXxx()` method.

Why HttpServlet is used? (Exam points)

- Automatically handles HTTP protocol details
- Provides session, cookies, and headers
- Reduces code (only override necessary methods)

Example (Exam-friendly)

java

 Copy code

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<h2>Hello Servlet</h2>");
    }
}
```

✓ 3. HttpServletRequest — Exam-Oriented Explanation

`HttpServletRequest` is used to read client request data in an HTTP request.

It extends `ServletRequest`.

Important Methods

1 `getParameter(String name)`

Reads form data

java

 Copy code

```
String user = req.getParameter("username");
```

2 `getHeader(String name)`

Reads request headers

java

 Copy code

```
String agent = req.getHeader("User-Agent");
```

3 `getCookies()`

Returns cookies

4 `getMethod()`

Returns the HTTP method
(GET/POST/PUT/DELETE)

5 `getRequestURL()` / `getRequestURI()`

Returns complete URL or path

6 `getSession()`

Returns an `HttpSession` object.

What `HttpServletRequest` provides?

- Form inputs
- Query string
- Cookies
- Header info
- Session management
- Client IP, host info

✓ 4. `HttpServletResponse` — Exam-Oriented Explanation

`HttpServletResponse` is used to **generate response** back to the client.

It extends `ServletResponse`.

Important Responsibilities

- Set response content type
- Send HTML or JSON output
- Add headers
- Send cookies
- Redirect the user

Important Methods

1 `setContentType(String type)`

java

```
res.setContentType("text/html");
```

2 `getWriter()`

Returns **PrintWriter** to send text/HTML.

3 `getOutputStream()`

Sends binary data (files, images).

4 `sendRedirect(String url)`

Redirects the client to another page.

5 `addCookie(Cookie c)`

Adds cookies to response.

Example

java

 Copy code

```
res.setContentType("text/html");  
PrintWriter out = res.getWriter();  
out.println("<h1>Response Output</h1>");
```

★ VERY Important Exam Questions

2 Marks / Short Answer

1. What is the Servlet interface?
 2. What is the role of `HttpServlet` class?
 3. What is the use of `HttpServletRequest`?
 4. What is the use of `HttpServletResponse`?
 5. List any two methods of `HttpServletRequest`.
 6. List any two methods of `HttpServletResponse`.
-

5 Marks Questions

1. Explain the methods of the Servlet interface.
 2. What is HttpServlet? Explain with its life cycle.
 3. Explain the purpose and methods of HttpServletRequest.
 4. Explain the role of HttpServletResponse with examples.
-

10 Marks / Long Answer Questions

1. Describe the Servlet interface in detail with method definitions and lifecycle.
 2. Explain HttpServlet, HttpServletRequest, and HttpServletResponse in detail.
 3. Differentiate between GenericServlet and HttpServlet with examples.
 4. Write a servlet program using HttpServletRequest and HttpServletResponse to process form data.
-

Viva / Oral Questions

1. Why do we override doGet() and doPost() instead of service()?
 2. Name differences between GET and POST.
 3. How does HttpServlet support the HTTP protocol?
 4. How do you read form parameters in a servlet?
 5. How do you send a redirect in a servlet?
-

If you want, I can also prepare:

- ✓ A comparison table of Servlet vs GenericServlet vs HttpServlet
- ✓ One-page short notes for exam revision
- ✓ Diagram-based life cycle explanation



Exception Handling
Servlet, DAO, POJO DB Layers
exam-oriented explanation with important exam
questions.

Below is a clear, exam-oriented explanation of
Exception Handling in Servlet, DAO, and POJO-DB Layers,
along with **important exam questions** for theory, viva, and assignments.

✓ 1. Exception Handling in Servlets — Exam-Oriented Explanation

In servlets, exceptions may occur due to:

- Invalid inputs
- I/O errors
- DB connection failures
- Null values
- Network problems

Servlet exception handling ensures that the application:

- Does **not crash**
 - Shows **friendly error messages**
 - Logs details for debugging
 - Maintains **application stability**
-

✓ Techniques for Exception Handling in Servlets

1 Using try-catch blocks inside doGet() / doPost()

Most common method.

java

Copy code

```
try {  
    // DB or business logic  
} catch (SQLException e) {  
    e.printStackTrace();  
    throw new ServletException("Database error!");  
}
```

2 Using sendError()

Sends HTTP error response.

java

 Copy code

```
res.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid Input");
```

3 Using error pages in web.xml

Centralized error handling.

xml

 Copy code

```
<error-page>
  <exception-type>java.lang.Exception</exception-type>
  <location>/error.jsp</location>
</error-page>
```

4 Logging Exceptions

Using:

- ServletContext.log()
- Log4j / SLF4J

java

 Copy code

```
getServletContext().log("Error occurred", e);
```

Key Points (Exam)

- Servlet should NOT expose internal errors to users
- Use centralized error pages
- Always log errors
- Wrap low-level exceptions using ServletException or IOException

2. Exception Handling in DAO Layer — Exam-Oriented Explanation

DAO (Data Access Object) contains code for database operations such as:

- insert
- update
- delete
- select

DAO interacts with JDBC → most exceptions are **SQLException**.

✓ Why exception handling is important in DAO?

- Database failures are common
 - Need to avoid app crash
 - Provide meaningful error messages
 - Maintain transaction integrity
-

✓ Techniques Used in DAO Layer

1 Catch SQLException and wrap in custom exception

DAO should NOT throw SQLException directly to servlet.

java

 Copy code

```
try {  
    // DB code  
} catch (SQLException e) {  
    throw new DAOException("DB Error: Unable to fetch data", e);  
}
```

2 Close JDBC resources in finally

Using try-with-resources.

java

 Copy code

```
try (Connection con = DBUtil.getConn();  
     PreparedStatement ps = con.prepareStatement(sql)) {  
} catch (SQLException e) {
```



```
...  
}
```

3 Use custom exception classes

Example:

java

 Copy code

```
public class DAOException extends Exception {  
    public DAOException(String msg, Throwable t) {  
        super(msg, t);  
    }  
}
```

4 Proper transaction handling

- Commit on success
- Rollback on failure

java

 Copy code

```
con.setAutoCommit(false);  
try {  
    // operations  
    con.commit();  
} catch (Exception e) {  
    con.rollback();  
}
```

Key Points (Exam)

- DAO layer isolates DB logic
- MUST convert SQLException → Custom app exception
- Must use try-with-resources
- Should NOT directly interact with UI layer

✓ 3. Exception Handling in POJO Layer — Exam-Oriented Explanation

POJO (Plain Old Java Object) contains:

- Fields
- Getters/Setters
- Constructors

POJOs usually **do not contain business logic**.

Therefore, exceptions in POJO classes are rare.

✓ When exceptions occur in POJO Layer?

1 Invalid data passed to setter

Example: negative age, empty email, etc.

2 Data conversion errors

(String → int, date parsing)

✓ How to handle exceptions in POJO layer?

1 Throw IllegalArgumentException for invalid data

java

Copy code

```
public void setAge(int age) {  
    if (age < 0) throw new IllegalArgumentException("Age cannot be negative");  
    this.age = age;  
}
```

2 Use validation methods

java

Copy code

```
public void validate() throws ValidationException {  
    if (name == null || name.isEmpty())
```

```
        throw new ValidationException("Name required");  
    }  
}
```

3 Avoid DB and business logic in POJO

POJO should remain clean & lightweight.

Key Points (Exam)

- POJO should not handle DB logic
- Exception handling is minimal
- POJOs can throw `IllegalArgumentException` for invalid fields
- Validation is done before passing to DAO layer

Overall Architecture (Exam Note)

pgsql

 Copy code

```
Servlet Layer → Controller  
↓               |  
POJO Layer → Data Carrier  
↓               |  
DAO Layer → Database Access
```

Exception Flow (Important)

vbnet

 Copy code

DAO throws `custom` exception → Servlet catches it → Shows user-friendly `error` → L



IMPORTANT EXAM QUESTIONS

2-Marks / Short Notes

1. What is the role of exception handling in servlets?

2. What is DAOException? Why is it used?
 3. Why should POJO classes avoid handling database exceptions?
 4. What is sendError() in HttpServletResponse?
 5. Write two advantages of custom exceptions.
-

5-Marks Questions

1. Explain exception handling techniques used in servlets.
 2. Explain how exceptions are handled in DAO layer with example.
 3. What is the need for custom exception classes in DAO?
 4. Write notes on exception handling in POJO classes.
-

10-Marks / Long Answer Questions

1. Explain exception handling in Servlet, DAO, and POJO layers with diagrams.
 2. Describe layered architecture and how exceptions propagate across layers.
 3. Write a program to demonstrate exception handling in a servlet using DAO and POJO layers.
-

Viva Questions

1. What is the difference between ServletException and IOException?
 2. Why do we wrap SQLException in a custom exception?
 3. What happens if exceptions are not handled in DAO layer?
 4. Why is try-with-resources preferred in JDBC?
 5. What is the best place to display user-friendly error messages?
-

If you want, I can also prepare:

- ✓ A diagram of Servlet–DAO–POJO Exception Flow
- ✓ A 10-mark full answer with diagram
- ✓ Layered architecture explanation in short notes



Session
Session Management
Session Tracking with exam-oriented explanation with
important exam questions.

Below is a **clear, exam-oriented explanation** of **Session, Session Management, and Session Tracking in Servlets**, along with **important exam questions (2M, 5M, 10M, viva)**.
Covers theory + examples + diagrams (in text) — perfect for university & interview preparation.

✓ 1. SESSION – EXAM-ORIENTED EXPLANATION

A **Session** is a period of interaction between a **client** and a **server**.

Since HTTP is **stateless**, the server does not remember previous requests.

To maintain user-specific data (cart items, login status), the server creates a **Session** object.

Key Points (Write in exam):

- Session = series of request–response interactions.
- Server identifies the user using a **Session ID (JSESSIONID)**.
- Session object stores user-specific data.
- Session ends when:
 - ✓ Timeout happens
 - ✓ User logs out
 - ✓ Session invalidated

Session Methods (memory tips for exam):

Method	Purpose
setAttribute(name, value)	Store data
getAttribute(name)	Retrieve data
invalidate()	Destroy session
getId()	Returns session ID

Method	Purpose
<code>getCreationTime()</code>	Timestamp
<code>setMaxInactiveInterval()</code>	Timeout

✓ 2. SESSION MANAGEMENT – EXAM-ORIENTED EXPLANATION

Session management means **maintaining state** (user data) across multiple requests.

Why Session Management?

- HTTP is stateless
- Needed for login systems, shopping carts, banking, dashboards, etc.

Techniques of Session Management

1. Cookies
2. URL Rewriting
3. Hidden Form Fields
4. Session Tracking using `HttpSession`
5. SSL Sessions (rare in exams)

✓ 3. SESSION TRACKING – EXAM-ORIENTED EXPLANATION

Session Tracking = Techniques used by server to identify user requests belonging to the same session.

Methods of Session Tracking (VERY IMPORTANT):

3.1 Cookies

- Data stored in user's browser.
- Server adds cookie using `response.addCookie()` .
- Client returns cookie using `request.getCookies()` .

Types

- **Persistent Cookie** – stored on disk
- **Non-persistent Cookie** – memory only
- **Secure Cookie** – sent only over HTTPS

Advantages

- Easy to use
- Automatically managed

Disadvantages

- Disabled cookies → no tracking
 - Security issues
-

3.2 URL Rewriting

Session ID is added at end of URL:

```
arduino
```

 Copy code

```
http://example.com/welcome;jsessionid=123ABC
```

Used when cookies are disabled.

Advantages

- Works even if cookies off

Disadvantages

- URL becomes long
 - Security risk
-

3.3 Hidden Form Fields

Session data stored as hidden `<input>` fields.

Example:

```
html
```

 Copy code

```
<input type="hidden" name="user" value="John">
```

Limitations

- Works only with form submissions
- Not for hyperlink navigation

3.4 HttpSession (Most important in exam)

Most popular tracking technique.

Example:

java

 Copy code

```
HttpSession session = request.getSession();
session.setAttribute("user", "John");
```

Advantages

- Most secure
- Stores Java objects
- Used for login + cart systems

Disadvantages

- Server memory consumption increases

 EXAM-ORIENTED SHORT NOTES

Difference between Cookie and HttpSession

Cookie	HttpSession
Stored in browser	Stored on server
Less secure	More secure
Limited data	Can store objects
Lightweight	Heavy (server memory)

 Exam-Friendly Code Example: HttpSession

java

 Copy code

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        String user = req.getParameter("username");

        HttpSession session = req.getSession();
        session.setAttribute("username", user);

        res.getWriter().println("Welcome " + user);
    }
}
```

Important Diagram (ASCII – write in exam)

arduino

 Copy code

```
Client ---- Request ----> Server
Client <--- Response ---- SessionID (JSESSIONID)
```

Next Request:

```
Client ---- Request + JSESSIONID ---> Server
Server identifies user and restores session
```

Most Important Exam Questions

2-Mark Questions

1. What is a session in Servlets?
2. What is session tracking?
3. List any two session tracking techniques.
4. Write difference between Cookie and Session.
5. What is URL rewriting?

! 5-Mark Questions

1. Explain HttpSession with example program.
2. Describe various session tracking techniques in servlets.
3. Explain cookies with advantages and disadvantages.
4. How does URL rewriting work?

! 10-Mark / Long Answer Questions

1. Explain session management in Servlets. Describe all techniques with examples.
2. Explain cookies, URL rewriting, and HttpSession in detail.
3. Write a servlet program that uses HttpSession to store and retrieve user information.
4. Compare all session tracking techniques with merits & demerits.

Viva Questions

1. Why is HTTP called stateless?
2. What is JSESSIONID?
3. Can session work if cookies are disabled?
4. What is session timeout?
5. Which technique is most secure and why?

If you want, I can also create:

-  One-page summary notes
-  Diagram-based revision sheet
-  Full unit-wise servlet notes for exam preparation



Cookies exam-oriented explanation with important exam questions.

Below is a **crisp, exam-oriented explanation of Cookies in Servlets**, followed by **important exam questions (2M, 5M, 10M, viva)**.

Useful for university theory exams, practicals, and interviews.



COOKIES — EXAM-ORIENTED EXPLANATION



What is a Cookie?

A **Cookie** is a small piece of data stored in the **client's browser** by the server.

It is used to **store user information** and helps in **session tracking**.

HTTP is **stateless**, so cookies help the server identify the same user between multiple requests.



Why Cookies? (Write in exam)

- HTTP does not maintain state
 - Cookies store user preferences, login info, browsing behavior
 - Used for personalization (themes, language, cart items)
 - Used for session tracking (JSESSIONID)
-



Types of Cookies

1. Non-Persistent Cookie (Session Cookie)

- Stored in memory
- Deleted when browser closes

2. Persistent Cookie

- Stored on disk
- Has expiry time

3. Secure Cookies

- Sent only on HTTPS

4. HttpOnly Cookies

- Not accessible through JavaScript → More secure
-



Cookie Class (Methods to write in exam)

Method	Description
<code>new Cookie(name, value)</code>	Create cookie
<code>setMaxAge(int seconds)</code>	Set expiry
<code>getName()</code>	Cookie name
<code>getValue()</code>	Cookie value
<code>response.addCookie(cookie)</code>	Send cookie to browser
<code>request.getCookies()</code>	Fetch cookies

Example 1 — Creating a Cookie (VERY IMPORTANT)

java

 Copy code

```
Cookie ck = new Cookie("user", "Yuvraj");
ck.setMaxAge(3600); // 1 hour
response.addCookie(ck);
```

Example 2 — Reading Cookies

java

 Copy code

```
Cookie[] cookies = request.getCookies();
for(Cookie c : cookies){
    if(c.getName().equals("user")) {
        out.println("User: " + c.getValue());
    }
}
```

Advantages of Cookies (Exam Points)

- No server memory usage
- Simple to implement
- Persistent storage

- Automatically handled by browser

Disadvantages of Cookies

- Users can disable cookies
- Limited size (4 KB)
- Security issues → stored in browser
- Only String values can be stored

Security Issues with Cookies

- Can be stolen using XSS
- Not encrypted
- Sensitive data (passwords) should never be stored
- Use Secure + HttpOnly flags
- Use HTTPS

Use Cases of Cookies

- Remember me (auto-login)
- Language preferences
- Shopping cart state
- Theme settings
- Analytics & tracking

COOKIES VS SESSION (Write in exam)

Cookies	Session
Stored in browser	Stored on server
Less secure	More secure
Lightweight	Uses server memory
Only strings	Stores objects

Cookies**Session**

Limited storage

Large storage possible



MOST IMPORTANT EXAM QUESTIONS

! 2-Mark Questions

1. What is a cookie?
2. List types of cookies.
3. What is the default lifespan of a cookie?
4. How do you create a cookie in a servlet?
5. What is the use of `setMaxAge()` ?

! 5-Mark Questions

1. Explain the life cycle of a cookie with example.
2. Write advantages and disadvantages of cookies.
3. Explain persistent and non-persistent cookies.
4. Write a servlet program to create and read cookies.

! 10-Mark / Long Answer Questions

1. Explain cookies in detail with types, advantages, disadvantages, and example programs.
2. Discuss the role of cookies in session tracking. Explain how cookies are created, sent, and retrieved by the server.
3. Compare Cookies and HttpSession with examples.
4. Explain cookie security issues and methods to secure cookies.



Viva Questions

1. Can cookies store objects?
2. Are cookies secure? Why?

3. What happens if cookies are disabled in browser?
4. Difference between persistent and session cookies?
5. How does server send a cookie to client?

If you want, I can also prepare:

-  A one-page last-minute revision sheet
-  Program-based questions
-  Diagram-based explanation of cookie flow



HttpSession
□ Request Dispatcher
□ Page Navigation
□ Complete Case study Servlet Based
exam-oriented explanation with important exam
questions.

Below is a **complete, exam-oriented explanation** of

- ✓ HttpSession
- ✓ RequestDispatcher
- ✓ Page Navigation
- ✓ Full Servlet-Based Case Study

with **important exam questions** (2M, 5M, 10M, Viva).

Perfect for **university exams, practicals, and interviews**.

1. HttpSession — Exam-Oriented Explanation

HttpSession is an interface used to **store user-specific data** on the server side.

HTTP is **stateless**, so session helps maintain **state information** across multiple requests.

◆ Features (Write these in exam):

- Server-side storage
- Session created using `request.getSession()`
- Each session has a unique **JSESSIONID**

- Stores objects using `setAttribute()`
- Auto-expires after inactivity
- Used in login/cart applications

◆ Common Methods

Method	Description
<code>getSession()</code>	Creates/returns session
<code>setAttribute(name, value)</code>	Stores data
<code>getAttribute(name)</code>	Retrieves data
<code>removeAttribute(name)</code>	Removes data
<code>invalidate()</code>	Destroys session
<code>getId()</code>	Returns session ID
<code>getMaxInactiveInterval()</code>	Timeout

◆ Example

java

 Copy code

```
HttpSession session = request.getSession();
session.setAttribute("uname", "Yuvraj");
```

✓ 2. RequestDispatcher — Exam-Oriented Explanation

`RequestDispatcher` is used for **page navigation** within server-side resources (servlets → JSP, servlet → servlet).

Two methods:

1. `forward()`

- Transfers request to another resource
- Browser **does NOT** know
- URL **does not** change
- Faster, used for MVC and form submission

java

```
RequestDispatcher rd = request.getRequestDispatcher("dashboard.jsp");
rd.forward(request, response);
```

2. include()

- Includes content of another resource
- Used for common layouts (header, footer)

java

Copy code

```
RequestDispatcher rd = request.getRequestDispatcher("header.jsp");
rd.include(request, response);
```



Difference: forward() vs sendRedirect()

forward()	sendRedirect()
Server-side	Client-side
URL unchanged	URL changes
Faster	Slower
Cannot redirect outside app	Can redirect anywhere



3. Page Navigation Techniques

1. RequestDispatcher (Forward/Include)

Used for internal navigation
e.g. servlet → JSP

2. sendRedirect()

Client-side redirection
Used for navigation to external URLs.

java

Copy code

```
response.sendRedirect("home.jsp");
```

3. Hyperlinks

Simple navigation, no server logic.

4. Form Submission

Navigation along with data.

★ 4. Complete Case Study — Servlet-Based Login System

A simple **Login Application** using

- ✓ Servlet
- ✓ HttpSession
- ✓ RequestDispatcher
- ✓ Page Navigation

Step 1: Login Form (login.html)

html

 Copy code

```
<form action="login" method="post">
  Username: <input type="text" name="uname"><br>
  Password: <input type="password" name="pass"><br>
  <input type="submit" value="Login">
</form>
```

Step 2: Login Servlet (LoginServlet.java)

java

 Copy code

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException, ServletException {
```

```
String u = req.getParameter("uname");
String p = req.getParameter("pass");

if(u.equals("admin") && p.equals("123")) {
    HttpSession session = req.getSession();
    session.setAttribute("user", u);

    RequestDispatcher rd = req.getRequestDispatcher("welcome");
    rd.forward(req, res);
}
else {
    res.sendRedirect("login.html");
}
}
```

Step 3: Welcome Servlet (WelcomeServlet.java)

java

 Copy code

```
@WebServlet("/welcome")
public class WelcomeServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        HttpSession session = req.getSession(false);

        if(session != null) {
            String user = (String) session.getAttribute("user");
            res.getWriter().println("Welcome, " + user);
        } else {
            res.sendRedirect("login.html");
        }
    }
}
```

Step 4: Logout Servlet

java

```
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        HttpSession session = req.getSession(false);
        if(session != null) session.invalidate();
        res.sendRedirect("login.html");
    }
}
```

What This Case Study Demonstrates

- HttpSession for login persistence
- RequestDispatcher for internal navigation
- sendRedirect for client-side navigation
- Page navigation using servlets
- Complete login → welcome → logout cycle

Perfect for exam answers and practicals.

IMPORTANT EXAM QUESTIONS

2-Marks

1. What is HttpSession ?
2. What is the use of forward() method?
3. What is RequestDispatcher?
4. Difference between forward() and include().
5. Why sendRedirect() is slower?

5-Marks

1. Explain HttpSession with methods.
2. Describe RequestDispatcher with example.
3. Explain difference between forward() and sendRedirect().

4. Write a servlet program to demonstrate session tracking.
 5. Explain page navigation techniques in servlets.
-

▲ - -